

NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE & AI

MIDTERM EXAMINATION (PAPER-BASED)**Course:** Data Analysis with Python (DSAI1005)**Topics:** NumPy (20%), Pandas (20%), Viz (20%), Code (40%)**Duration:** 60 minutes — **Exam Code:** 301-EN*(No personal laptops, textbooks, or unauthorized notes allowed)***Full Name:** **Student ID:****Class / Cohort:** **Exam Room:** **Seat No:****MULTIPLE-CHOICE ANSWER SHEET** (*Write capital letters A, B, C, or D into corresponding boxes*):

Question	1	2	3	4	5	6	7	8	9	10	11	12
Answer												

Part I (60%)	Part II (40%)	TOTAL
Examiner 1:		
Examiner 2:		

PART I: MULTIPLE-CHOICE QUESTIONS (6.0 pts — 12 questions × 0.5 pt each)**Section A: Numerical Computing with NumPy (2.0 pts — 20% of exam)**

1. Consider the following NumPy array initialization and arithmetic expression:

```
import numpy as np
a = np.linspace(0, 10, 5)
b = np.arange(1, 6)
res = a + b * 2
```

What is the resulting array `res`?

- A. [2., 6.5, 11., 15.5, 20.] C. [2, 7, 11, 16, 20]
- B. [2., 4.5, 9., 13.5, 18.] D. [0., 4.5, 11., 15.5, 22.]

2. In NumPy, basic slicing creates an array
- view*
- rather than an independent memory copy. Consider:

```
m = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
v = m[:, -1, 1]
v[0] = 999
print(m[2, 1], v[2])
```

What values are printed to the console?

- A. 80 20 B. 999 20 C. 80 999 D. 999 80

3. Array
- A*
- has shape (4, 1, 6) and array
- B*
- has shape (3, 1). Under NumPy broadcasting rules, what is the shape of the evaluated array from the expression
- `C = A * B + np.zeros(6)`
- ?

- A. (4, 3, 6) B. (4, 1, 6) C. (3, 4, 6) D. Raises ValueError

4. Consider the following conditional filtering and axis reduction snippet:

```
x = np.array([[12, 25, 8], [30, 15, 22]])
filtered = np.where((x % 2 == 0) & (x > 10), x * 2, 0)
print(np.sum(filtered, axis=0))
```

What output array is displayed?

- A. [84, 0, 44] B. [24, 60] C. [84, 50, 44] D. [24, 0, 44]

Section B: Tabular Data Processing with Pandas (2.0 pts — 20% of exam)

5. When performing arithmetic addition between two Pandas Series S_1 and S_2 with non-identical string index labels:

```
import pandas as pd
s1 = pd.Series([10, 20, 30], index=['Hanoi', 'HCMC', 'Da Nang'])
s2 = pd.Series([5, 15, 25], index=['HCMC', 'Can Tho', 'Hanoi'])
res = s1.add(s2, fill_value=0)
```

What is the value of `res['Da Nang']` and `res['Can Tho']` in the resulting Series?

- A. Both evaluate to NaN. C. An `IndexMismatchError` exception is raised.
 B. `res['Da Nang'] = 30.0` and `res['Can Tho'] = 15.0`. D. `res['Da Nang'] = 0.0` and `res['Can Tho'] = 0.0`.

6. Consider the sales performance DataFrame `df`:

```
df = pd.DataFrame({
    'Revenue': [150, 280, 310, 450],
    'Cost': [100, 190, 200, 300]
}, index=['Q1', 'Q2', 'Q3', 'Q4'])
loc_slice = df.loc['Q2':'Q4', 'Revenue']
iloc_slice = df.iloc[1:3, 0]
```

How many elements are contained in `loc_slice` and `iloc_slice` respectively?

- A. `loc_slice`: 3 elements; `iloc_slice`: 2 elements C. `loc_slice`: 3 elements; `iloc_slice`: 3 elements
 B. `loc_slice`: 2 elements; `iloc_slice`: 2 elements D. `loc_slice`: 2 elements; `iloc_slice`: 3 elements

7. Given the multi-level aggregation pipeline on retail transactions:

```
orders = pd.DataFrame({
    'Region': ['North', 'South', 'North', 'South', 'North'],
    'Category': ['Food', 'Food', 'Tech', 'Tech', 'Food'],
    'Sales': [100, 150, 300, 250, 200]
})
summary = orders.groupby(['Region', 'Category'])['Sales'].agg(['sum', 'count'])
print(summary.loc(['North', 'Food'], 'sum'))
```

What value is printed to the console?

- A. 100 B. 300 C. 2 D. 450

8. You execute a Left Merge between table A ($N = 500$ rows) and table B ($M = 80$ rows) on key 'store_id':

```
merged = pd.merge(A, B, on='store_id', how='left')
```

Assuming 'store_id' is unique in table B, but 30 rows in table A reference a `store_id` that does not exist in table B. Which statement is true regarding `merged`?

- A. `merged` contains exactly 470 rows, dropping the 30 unmatched entries.
 B. `merged` contains exactly 500 rows, and columns originating from table B will have NaN for those 30 rows.
 C. `merged` contains exactly 80 rows, matching only the intersection.
 D. A `KeyError` is thrown because all foreign keys must exist in the right table.

Section C: Data Visualization with Matplotlib & Seaborn (2.0 pts — 20% of exam)

9. In Matplotlib's Object-Oriented interface, consider the canvas creation:

```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
```

Which statement correctly accesses and configures the subplot located in the **first row, second column**?

- A. `axes[0, 1].set_title("Branch Q2")`
 - B. `axes[1, 2].set_title("Branch Q2")`
 - C. `axes[1, 0].title = "Branch Q2"`
 - D. `plt.subplot(1, 2).set_title("Branch Q2")`
10. In exploratory data analysis, when analyzing continuous financial transaction amounts containing substantial skewness and extreme high-value outliers, which visualization tool is most appropriate for observing the median, interquartile range (*IQR*), and whisker thresholds simultaneously?
- A. `sns.barplot()`
 - B. `sns.boxplot()`
 - C. `sns.countplot()`
 - D. `sns.lineplot()`
11. Consider the following Seaborn relational plotting statement:

```
import seaborn as sns
sns.scatterplot(data=df, x='Marketing_Spend', y='Revenue', hue='Channel', style='Tier')
```

What role do the parameters `hue='Channel'` and `style='Tier'` play in the resulting visualization?

- A. `hue` maps channel categories to distinct point colors; `style` maps tier categories to distinct marker shapes (glyphs).
 - B. `hue` adjusts point opacity; `style` sets the background CSS layout.
 - C. `hue` partitions data into separate subplots; `style` specifies the font style.
 - D. Both parameters are synonymous and map data to line width.
12. Given the Pearson correlation matrix computation and visualization:

```
corr_matrix = df[['Age', 'Income', 'Spending_Score', 'Tenure']].corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.tight_layout()
```

What is the primary function of `annot=True` and `plt.tight_layout()` respectively?

- A. `annot=True` adds interactive tooltips; `plt.tight_layout()` compresses the image to a low-resolution thumbnail.
- B. `annot=True` writes numeric correlation coefficients inside each cell; `plt.tight_layout()` adjusts subplot margins to prevent label clipping.
- C. `annot=True` filters out correlations near zero; `plt.tight_layout()` converts the plot to black and white.
- D. `annot=True` prints p-values; `plt.tight_layout()` locks axes aspect ratios to 1:1.

PART II: APPLIED CODING — FILL IN THE BLANKS (4.0 pts)

Problem 1: Quantitative E-Commerce Performance Pipeline (2.0 pts) — [NumPy & Pandas]

Scenario: An enterprise processes transactional batches. Complete the vectorized calculation and tabular aggregation pipeline by filling in the missing code expressions for [BLANK 1] through [BLANK 5].

```
import numpy as np
import pandas as pd

# Raw transactional logs: 5 orders across 4 numerical metrics
# Columns: [0: unit_price, 1: quantity, 2: discount_rate (0 to 1), 3: shipping_cost]
tx_data = np.array([
    [50000.0, 3.0, 0.10, 15000.0], [120000.0, 1.0, 0.00, 20000.0],
    [45000.0, 4.0, 0.05, 10000.0], [300000.0, 2.0, 0.20, 25000.0],
    [80000.0, 5.0, 0.15, 15000.0]
])

# Step 1: Vectorized Net Revenue: (unit_price * quantity * (1 - discount_rate)) - shipping_cost
net_revenue = _____ # [BLANK 1]

# Step 2: Categorize orders into "Wholesale" if net_revenue >= 250000 or quantity >= 4, else "Retail"
order_tier = _____ # [BLANK 2]

# Step 3: Integrate into DataFrame and merge with customer profile records
orders_df = pd.DataFrame({
    'order_id': ['001', '002', '003', '004', '005'],
    'cust_id': ['C10', 'C20', 'C10', 'C30', 'C20'],
    'net_rev': net_revenue, 'tier': order_tier
})
cust_df = pd.DataFrame({
    'cust_id': ['C10', 'C20', 'C30', 'C40'],
    'city': ['Hanoi', 'HCMC', 'Da Nang', 'Hanoi'],
    'status': ['VIP', np.nan, 'Regular', 'VIP']
})

# Merge orders_df with cust_df on 'cust_id' (retain all orders, perform a left merge):
merged = orders_df.merge(_____) # [BLANK 3]
merged['status'] = merged['status'].fillna('Standard')

# Step 4: Multi-level aggregation by ['city', 'tier']: calculate sum and count on 'net_rev'
summary = merged.groupby(_____) # [BLANK 4]

# Step 5: Sort the summarized table by total revenue descending
final_sorted = summary.sort_values(by=_____) # [BLANK 5]
```

Student Answer Sheet for Problem 1 (Write exact Python expressions):

Blank Number	Your Exact Code Expression
[BLANK 1] (0.4 pt) <i>Vectorized Net Revenue formula</i>	
[BLANK 2] (0.4 pt) <i>np.where categorization</i>	
[BLANK 3] (0.4 pt) <i>orders_df.merge(...) arguments</i>	
[BLANK 4] (0.4 pt) <i>groupby(...) & .agg(...) arguments</i>	
[BLANK 5] (0.4 pt) <i>sort_values(...) arguments</i>	

Problem 2: Multi-Channel Sales Dashboard & Risk Analytics (2.0 pts) — [Pandas & Viz]

Scenario: You are constructing an executive multi-panel analytics dashboard comparing sales distributions across payment channels and identifying risk anomalies. You are provided with a pre-loaded DataFrame `df_sales`:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Schema of df_sales:
# 'order_date': string formatted ('2026-09-01', ...)
# 'channel': string ('Online', 'Store', 'Partner')
# 'payment': string ('Credit Card', 'Cash', 'Bank Transfer')
# 'amount': float (order value in VND)
# 'is_fraud': int (1 for suspicious/fraudulent, 0 for legitimate)
```

Part A: Canvas Layout & Grouped Categorical Boxplot (1.0 pt)

Complete the code to set up a side-by-side subplot canvas and construct a grouped Seaborn boxplot displaying transaction amounts by payment method, stratified by fraud status.

```
# Step 1: Initialize figure and a 1x2 subplot grid (1 row, 2 columns) with figure size (14, 5) inches
fig, axes = _____ # [BLANK 6]

# Step 2: Plot a grouped Boxplot on axes[0]
# - Data source: df_sales; X-axis: 'payment'; Y-axis: 'amount'
# - Stratification hue: 'is_fraud'; Color palette: 'Set2'; Target axes: axes[0]
sns.boxplot(
    _____ # [BLANK 7]
)
axes[0].set_title("Transaction Amount Distribution by Payment Method", fontsize=12)
axes[0].set_xlabel("Payment Method", fontsize=10)
axes[0].set_ylabel("Order Value (VND)", fontsize=10)
```

Student Answer Sheet for Problem 2 — Part A:

Blank Number	Your Exact Python Code Expression
[BLANK 6] (0.4 pt) <i>plt.subplots setup</i>	
[BLANK 7] (0.6 pt) <i>sns.boxplot parameters</i>	

Part B: Time-Series Revenue Trajectory & Moving Average (0.6 pt)

On the second subplot (`axes[1]`), you must convert order dates to datetime, aggregate weekly total revenue, compute a 4-week rolling moving average to smooth short-term variance, plot both series, and adjust subplot spacing.

```
# Convert 'order_date' to datetime and set as DataFrame index:
df_sales['order_date'] = pd.to_datetime(df_sales['order_date'])
df_sales = df_sales.set_index('order_date')

# Step 3: Resample weekly ('W'), aggregate sum of 'amount', and calculate 4-week rolling mean:
weekly_sales = df_sales['amount'].resample('W').sum()
rolling_trend = weekly_sales._____ # [BLANK 8]

# Step 4: Plot both raw weekly sales and smoothed rolling trend on axes[1]
axes[1].plot(weekly_sales.index, weekly_sales.values, label='Weekly Actual', color='steelblue', alpha=0.6)
axes[1].plot(weekly_sales.index, rolling_trend.values, label='4-Week Trend', color='crimson', linewidth=2)
axes[1].set_title("Weekly Revenue Trajectory & 4-Week Moving Average", fontsize=12)
axes[1].set_xlabel("Week Ending Date", fontsize=10)
axes[1].set_ylabel("Total Revenue (VND)", fontsize=10)
axes[1].legend()
axes[1].grid(True, linestyle='--', alpha=0.5)

# Step 5: Automatically adjust padding to prevent overlapping labels and titles across subplots:
_____ # [BLANK 9]
```

Student Answer Sheet for Problem 2 — Part B:

Blank Number	Your Exact Python Code Expression
[BLANK 8] (0.4 pt) <i>rolling mean call</i>	
[BLANK 9] (0.2 pt) <i>layout padding call</i>	

Part C: Analytical Interpretation & Executive Decision-Making (0.4 pt)

Answer the following two short questions based on the visualization dashboard constructed above:

1. In the boxplot on `axes[0]`, how can a risk analyst visually differentiate between typical legitimate transactions and fraudulent outliers in terms of the interquartile range (*IQR*) and whisker boundaries?
2. Why is plotting a 4-week rolling trend line on `axes[1]` analytically superior to observing only erratic week-to-week revenue spikes when forecasting quarterly revenue?

Executive Decision Answers (Write concise answers below):

1. *Outlier Identification:*

2. *Forecasting Advantage of Rolling Average:*

— END OF EXAMINATION PAPER —